Link del taller explicando:
https://drive.google.com/file/d/1xRYvSzNUTYcL1-tO_XxcuKVb4DJeawg4/view?usp=sharing

Instituto Tecnológico de Costa Rica
Escuela Ingeniería en Computadores
Curso: CEXXXX Nombre_curso

Documentación técnica
Nombre_proyecto

Estudiantes:

Nombre_estudiante_1
Nombre_estudiante_2
Nombre_estudiante_3

Profesor:

Nombre_profesor

Semestre I 2026

0. Indice

0. Indice 2
1. Introducción 4
1.1. Propósito del documento 4
1.2. Descripción del proyecto 4
1.3 Objetivos y Alcance 4
1.4 Definiciones, Acrónimos y Abreviaturas 4
2. Descripción General del Sistema 5
2.1. Perspectiva del Producto 5
2.2. Funcionalidades principales 5
2.3. Restricciones del sistema, Supuestos y Dependencias 5
3. Requisitos del Sistema 7
3.1. Requisitos Funcionales 7
3.2. Requisitos No Funcionales 7
4. Diseño de la Solución 8
4.1. Visión General de la Arquitectura 8
4.2. Descripción de Componentes 8
4.4. Diagrama de Clases y Modelo de Datos 9
4.5. Diagrama de Flujo y Secuencia 9
4.6. Decisiones de Diseño y Justificación 9
5. Implementación y Validación de Calidad 11
5.1. Estructura del repositorio 11
5.2. Entorno de Desarrollo, Guía de instalación y Ejecución 11
5.3. Estrategias y Casos de Prueba 12
5.4. Resultados de pruebas y problemas encontrados 12
6. Declaración de uso de Inteligencia Artificial 14
6.1. Herramientas de IA utilizadas 14
6.2. Herramientas de IA utilizadas 14
6.3. Declaración de Responsabilidad y Autoría 15
7. Conclusión 16
8. Bibliográfia 17

1. Introducción

Esta sección presenta el contexto general del proyecto, la justificación de su desarrollo y los objetivos que se persiguen. Permite al lector comprender el problema que motiva la solución de software propuesta.

1.1. Propósito del documento

Describe a quién va dirigido este documento y qué busca comunicar. En este apartado deben escribir el propósito del documento, además de indicar cuál es la audiencia esperada a alcanzar.
Ejemplo: 'El presente documento describe la arquitectura, diseño e implementación del sistema/programa/proyecto [nombre], destinado a [audiencia].

1.2. Descripción del proyecto

Explica brevemente qué es el sistema, qué hace y en qué contexto opera. Responde las preguntas: ¿qué?, ¿para qué? y ¿para quién?. En este apartado se describe el sistema de software: nombre, tipo de aplicación (web, móvil, escritorio, API, etc.), dominio de aplicación y usuario final previsto.
Ejemplo:

1.3 Objetivos y Alcance

Se describe el alcance esperado para el cual se proponen el o los objetivos principales y, por cada uno de estos, los objetivos específicos. Define los límites funcionales del sistema.

1.4 Definiciones, Acrónimos y Abreviaturas

Se listan todos los términos técnicos, siglas y abreviaturas utilizados en el documento.

Esta sección se utiliza principalmente en documentación técnica de proyectos grandes de software. Sección inspirada en documentación de Google Cloud

2. Descripción General del Sistema

Esta sección ofrece una visión de alto nivel del sistema: sus características generales, las restricciones bajo las cuales opera y los supuestos que guiaron su diseño.

2.1. Perspectiva del Producto

Describe el entorno en el que opera y cómo se relaciona con otros sistemas. Se debe describir si el sistema es autónomo o si interactúa con otros sistemas externos. Mencionar plataformas relacionadas, APIs de terceros o sistemas heredados.

2.2. Funcionalidades principales

Enumera las funcionalidades de alto nivel del sistema (no detalle de implementación). Puede representarse como una lista o diagrama de casos de uso.
Ejemplo:

- [Función principal 1]: [Descripción breve]
- [Función principal 2]: [Descripción breve]

2.3. Restricciones del sistema, Supuestos y Dependencias

Primero, se listan las limitaciones que condicionan el diseño o implementación: tecnológicas, regulatorias, de rendimiento, de tiempo, etc.
Ejemplo:

- [Restricción 1]: Ej. El sistema debe ser compatible con navegadores modernos (Chrome, Firefox, Edge).
- [Restricción 2]: Ej. El tiempo de respuesta no debe superar los 3 segundos en condiciones normales.

Seguidamente, se listan y documentan los supuestos que se asumieron como verdaderos durante el diseño, y las dependencias externas del sistema. Estos, con el fin de respetar las restricciones propuestas.
Ejemplo:
Supuestos:

- [Supuesto 1]: Ej. Los usuarios finales cuentan con acceso a internet estable.
- [Supuesto 2]: Ej. Se dispone de un servidor con las especificaciones mínimas indicadas.

Dependencias:

- [Dependencia 1]: Ej. Librería o framework X, versión Y.
- [Dependencia 2]: Ej. Servicio externo Z (API de pagos, correo, etc.).

3. Requisitos del Sistema

Esta sección documenta de forma estructurada todos los requisitos que el sistema debe cumplir.

3.1. Requisitos Funcionales

Describe las capacidades y comportamientos que el sistema debe tener. Usa identificadores únicos (RF-XXX) para cada requisito. Incluye descripción, prioridad y criterio de aceptación.
Ejemplo:

ID	Descripción	Prioridad	Criterio de Aceptación
RF-001	[Descripción del requisito funcional 1]	Alta	[Condición que debe cumplirse para dar el requisito por satisfecho]
RF-002	[Descripción del requisito funcional 2]	Media	[Criterio de aceptación]
RF-003	[Descripción del requisito funcional 3]	Baja	[Criterio de aceptación]

3.2. Requisitos No Funcionales

Describe atributos de calidad del sistema: rendimiento, seguridad, disponibilidad, escalabilidad, usabilidad, etc. Usa identificadores RNF-XXX.
Ejemplo:

ID	Categoría	Descripción	Métrica / Valor Esperado
RNF-001	Rendimiento	El sistema debe responder a las solicitudes del usuario en tiempo aceptable.	Tiempo de respuesta < 3 segundos en condiciones normales.
RNF-002	Seguridad	Los datos de los usuarios deben estar protegidos.	Autenticación con contraseña segura y cifrado de datos en tránsito (HTTPS).
RNF-003	Usabilidad	La interfaz debe ser intuitiva para usuarios no técnicos.	El 80% de los usuarios completan las tareas principales sin ayuda.

4. Diseño de la Solución

Este capítulo describe las decisiones de diseño tomadas para dar respuesta a los requisitos del sistema. Se documenta la arquitectura, los componentes, el modelo de datos y los flujos principales.

Se usó de referencia y se siguen las convenciones de diseño de Microsoft Azure Architecture Center y Google Cloud Architecture Framework.

4.1. Visión General de la Arquitectura

Presenta el estilo arquitectónico elegido (monolítico, microservicios, cliente-servidor, MVC, etc.) y justifica la decisión. Incluye un diagrama de arquitectura. Por lo que, se describe el patrón o estilo arquitectónico seleccionado y se justifica por qué fue la opción más adecuada para este proyecto considerando los requisitos y restricciones.

Se debe incluir el Diagrama de Arquitectura del Sistema (Capa de Presentación, Lógica de Negocio, Datos e Integraciones). Herramientas sugeridas: draw.io, Lucidchart.

4.2. Descripción de Componentes

Describe cada componente o módulo del sistema: qué hace, qué tecnología usa, qué interfaces expone y qué dependencias tiene.
Ejemplo:

Componente	Responsabilidad	Tecnología	Dependencias
Frontend	Interfaz de usuario y experiencia visual.	[Ej. React, Vue, Angular]	API REST del Backend
Backend / API	Lógica de negocio y exposición de endpoints.	[Ej. Node.js, Django, Spring]	Base de Datos, Servicios externos
Base de Datos	Persistencia y gestión de datos.	[Ej. PostgreSQL, MongoDB]	Backend
[Componente]	[Responsabilidad]	[Tecnología]	[Dependencias]

4.3. Stack Tecnológico

Lista todas las tecnologías, frameworks, librerías y herramientas utilizadas en el proyecto.
Ejemplo:

Categoría	Tecnología	Versión	Propósito
Lenguaje (Frontend)	[Ej. JavaScript / TypeScript]	[X.X]	Lógica del lado cliente
Framework Frontend	[Ej. React]	[X.X]	Construcción de UI
Lenguaje (Backend)	[Ej. Python]	[X.X]	Lógica de negocio
Framework Backend	[Ej. FastAPI]	[X.X]	Exposición de endpoints REST

4.4. Diagrama de Clases y Modelo de Datos

Describa la estructura de clases que se implementó y preséntelo en un diagrama de clases. Si se tiene una Base de Datos, describa la estructura de la base de datos: entidades, atributos, relaciones y restricciones. Igualmente, presente esta estructura en un diagrama Entidad-Relación (ER).

4.5. Diagrama de Flujo y Secuencia

Incluye diagramas que representan los flujos de proceso más importantes (UML: diagramas de secuencia, actividades o casos de uso). Documenta al menos los flujos críticos del negocio.

Se debe describir brevemente cada diagrama: qué flujo representa y cuáles son los actores o sistemas involucrados.

4.6. Decisiones de Diseño y Justificación

Documenta las decisiones de diseño relevantes (ADR - Architecture Decision Records). Por cada decisión indica el contexto, las alternativas evaluadas, la opción elegida y la justificación.

Este formato es utilizado en Google y Microsoft.

Decisión	Alternativas Evaluadas	Opción Elegida	Justificación
Base de datos	MySQL, PostgreSQL, MongoDB	[Opción elegida]	[Razón técnica o de negocio]
Arquitectura	Monolítica, Microservicios, Serverless	[Opción elegida]	[Razón técnica o de negocio]
[Decisión]	[Alternativas]	[Opción elegida]	[Justificación]

5. Implementación y Validación de Calidad

Describe la estructura del repositorio de código, el entorno de desarrollo, el proceso de construcción del proyecto y los pasos necesarios para su despliegue. Además para la validación de calidad se documenta la estrategia de pruebas del sistema, los resultados obtenidos, los problemas encontrados que tuvieron solución y los que no se pudieron resolver.

Se fundamenta en los estándares ISO/IEC 25010 (calidad de software), las guías de pruebas de Microsoft y la metodología de testing de Google SRE.

5.1. Estructura del repositorio

Describe la organización de carpetas y archivos del proyecto. Explica qué contiene cada directorio principal.
Ejemplo:

```
/proyecto-raiz  /src           → Código fuente principal  /tests         → Pruebas
unitarias e integración  /docs          → Documentación del proyecto  /config        →
Archivos de configuración  /scripts       → Scripts de automatización  README.md
→ Descripción general y guía de inicio rápido
```

5.2. Entorno de Desarrollo, Guía de instalación y Ejecución

Lista los prerrequisitos de software necesarios para ejecutar el proyecto en un entorno local de desarrollo. Además, se proporcionan instrucciones paso a paso para que cualquier persona pueda ejecutar el proyecto desde cero. Incluye comandos exactos.
Ejemplo:

- Entorno:
  - Sistema Operativo: [Windows 10+, macOS 12+, Ubuntu 20.04+]
  - Lenguaje / Runtime: [Ej. Node.js 18+ / Python 3.10+]

- Guía de instalación y ejecución:

```
# 1. Clonar el repositoriogit clone
https://github.com/[usuario]/[repositorio].gitcd [repositorio]# 2.
Instalar dependencias[npm install  /  pip install -r requirements.txt]#
3. Configurar variables de entornocp .env.example .env# Editar .env con
los valores correspondientes# 4. Inicializar la base de datos[comando de
migración]# 5. Ejecutar el servidor[npm start  /  python app.py]
```

5.3. Estrategias y Casos de Prueba

Describe la estrategia general de pruebas adoptada: tipos de prueba, herramientas utilizadas, cobertura, objetivo y responsables. Adicionalmente, se definen y documentan los casos de prueba más representativos con identificador, precondiciones, pasos, resultado esperado y resultado obtenido.
Ejemplo:

- Estrategias de pruebas:

Tipo de Prueba	Descripción	Herramienta	Cobertura Objetivo
Pruebas Unitarias	Verificación de funciones y métodos individuales de forma aislada.	[Jest / Pytest / JUnit]	>= 70% del código
Pruebas Funcionales (E2E)	Validación del comportamiento del sistema desde la perspectiva del usuario final.	[Cypress / Playwright / Selenium]	Casos de uso principales
Pruebas de Rendimiento	Medición del comportamiento bajo carga normal y máxima.	[k6 / JMeter / Locust]	Tiempo de respuesta < 3s
[Tipo de Prueba]	[Descripción]	[Herramienta]	[Cobertura]

- Casos de prueba:

ID	Descripción	Pasos	Resultado Esperado	Resultado Obtenido	Estado
CP-001	[Inicio de sesión exitoso]	1. Abrir app 2. Ingresar credenciales válidas 3. Clic en 'Ingresar'	El sistema redirige al dashboard.	[Resultado real]	✅ Pasó
CP-002	[Inicio de sesión fallido]	1. Ingresar credenciales incorrectas 2. Clic en 'Ingresar'	El sistema muestra un mensaje de error.	[Resultado real]	✅ Pasó
CP-003	[Descripción]	[Pasos]	[Esperado]	[Obtenido]	[Estado]

5.4. Resultados de pruebas y problemas encontrados

Resume los resultados de las pruebas ejecutadas. Presenta métricas de cobertura, problemas encontrados y resueltos. Puede añadir imágenes del funcionamiento del programa.

Ejemplo:

- Resultados:
    - Casos de prueba exitosos:
        - 100% de las pruebas unitarias funcionaron
        - Tiempo de ejecución-perfilado
    - Imágenes del programa corriendo

Problemas encontrados:

- Problemas Solucionados: Se describe el problema, cómo se generó y cuál fue la solución implementada.
- Problemas no solucionados: Se describe el problema, cómo se generó y cuál fue la razón por la que no se solucionó.

6. Declaración de uso de Inteligencia Artificial y herramientas

Principio de Transparencia Académica
El uso de herramientas de Inteligencia Artificial en proyectos académicos requiere declaración explícita y responsable. Esta sección documenta de forma honesta cómo, cuándo y para qué se utilizaron herramientas de IA en el desarrollo de este proyecto, siguiendo los principios de integridad académica

6.1. Herramientas de IA utilizadas

Lista todas las herramientas de IA utilizadas durante el desarrollo. Sé específico con el nombre, versión (si aplica) y propósito.
Ejemplo:

Herramienta de IA	Tipo	Período de Uso	Propósito
[Ej. ChatGPT / GPT-4]	Modelo de lenguaje (LLM)	[Mes/Año]	Generación de ideas, redacción de comentarios de código, resolución de dudas técnicas.
[Ej. GitHub Copilot]	Asistente de código	[Mes/Año]	Autocompletado y sugerencias de código durante el desarrollo.
[Ej. Claude]	Modelo de lenguaje (LLM)	[Mes/Año]	Revisión de documentación y ayuda con la depuración.
[Herramienta]	[Tipo]	[Período]	[Propósito]

6.2. Descripción del Uso de Inteligencia Artificial

Para cada herramienta o uso significativo de IA, describe con precisión en qué parte del proyecto se aplicó y cómo fue revisado y validado por el equipo.

Se debe indicar en qué apartado específico se utilizó:

- Generación y asistencia en código
- Redacción y documentación
- Investigación y resolución de problemas
- Diseño

6.3. Declaración de Responsabilidad y Autoría

Esta declaración debe ser redactada y firmada por todos los integrantes del equipo. Afirma que el trabajo final es producto del esfuerzo intelectual del equipo y que la IA fue utilizada como herramienta de apoyo.

Ejemplo:

DECLARACIÓN DE AUTORÍA Y RESPONSABILIDAD

Los suscritos, integrantes del equipo de trabajo del proyecto "[Nombre del Proyecto]", declaramos que:

1. El diseño, análisis, desarrollo y conclusiones del presente proyecto son resultado de nuestro trabajo intelectual y esfuerzo como equipo.
2. Las herramientas de Inteligencia Artificial fueron utilizadas exclusivamente como apoyo en tareas específicas descritas en este capítulo, no como sustituto de nuestro trabajo.
3. Todo el contenido generado por IA fue revisado, comprendido, validado y, cuando fue necesario, corregido por los integrantes del equipo.
4. Asumimos plena responsabilidad académica sobre el contenido total de este proyecto y su documentación.
5. El uso de IA se realizó conforme a las políticas de la institución educativa y los lineamientos del docente responsable.

7. Conclusión

Redactar 3-5 párrafos o puntos concluyentes sobre el proyecto.

Responder: ¿Se cumplieron los objetivos planteados? ¿Qué retos se superaron? ¿Qué aprendizajes técnicos y de trabajo en equipo obtuvo el grupo?

Además, se puede mencionar posibles mejoras a futuro en proyectos similares.

8. Bibliográfia

[1] IEEE, *IEEE Recommended Practice for Software Requirements Specifications*, IEEE Std 830-1998, IEEE Computer Society, New York, NY, USA, 1998. doi: 10.1109/IEEESTD.1998.88286

[2] ISO/IEC, *Systems and Software Engineering — Systems and Software Quality Requirements and Evaluation (SQuaRE) — System and Software Quality Models*, ISO/IEC 25010:2011, International Organization for Standardization, Geneva, Switzerland, 2011.

[3] IEEE, *IEEE Standard for Information Technology — Systems Design — Software Design Descriptions*, IEEE Std 1016-2009, IEEE Computer Society, New York, NY, USA, 2009. doi: 10.1109/IEEESTD.2009.5167255

[4] Google LLC, *Google Cloud Architecture Framework*, Google Cloud Documentation, 2024. [En línea]. Disponible en: https://cloud.google.com/architecture/framework

[5] Microsoft Corporation, *Azure Well-Architected Framework*, Microsoft Learn Documentation, 2024. [En línea]. Disponible en: https://learn.microsoft.com/en-us/azure/well-architected/

[6] Microsoft Corporation, *Microsoft Writing Style Guide*, Microsoft Docs, 2024. [En línea]. Disponible en: https://learn.microsoft.com/en-us/style-guide/welcome/

[7] Google LLC, *Google Developer Documentation Style Guide*, Google Developers, 2024. [En línea]. Disponible en: https://developers.google.com/style

[8] ISO/IEC/IEEE, *Systems and Software Engineering — Life Cycle Processes — Requirements Engineering*, ISO/IEC/IEEE 29148:2018, International Organization for Standardization, Geneva, Switzerland, 2018. doi: 10.1109/IEEESTD.2018.8559686

[9] M. Nygard, *Architecture Decision Records (ADRs)*, ThoughtWorks Technology Radar, 2017. [En línea]. Disponible en: https://adr.github.io

[10] Google LLC, *Site Reliability Engineering: How Google Runs Production Systems*, O'Reilly Media, Sebastopol, CA, USA, 2016. [En línea]. Disponible en: https://sre.google/sre-book/table-of-contents/